Bahçeşehir University

Faculty of Engineering & Natural Sciences

Department of Artificial Intelligence Engineering

Development and Evaluation of an MLflow-based ML Lifecycle Management System

Telco Customer Churn Prediction

Course: AIN-3009 — MLOps (Term Project)
Author: Mohamed Dribika
Student number: 2280197
Academic year: 2025–2026

ABSTRACT

This project designs and implements an end-to-end machine-learning lifecycle management system on the IBM Telco Customer Churn dataset, using MLflow as the platform of record. The five lifecycle stages specified by the assignment — experiment tracking, model training and tuning, model deployment, performance monitoring, and the model registry — are each implemented as a separate Python module and verified end-to-end. Three baseline classifiers are compared, a tree-structured Parzen estimator search produces the best test ROC-AUC of **0.8457**, the winning model is registered with *staging* → *production* alias transitions, served via MLflow's built-in scoring server, and monitored against three simulated production batches that demonstrate the limit of input-distribution drift detection when concept drift is present.

1. Problem definition

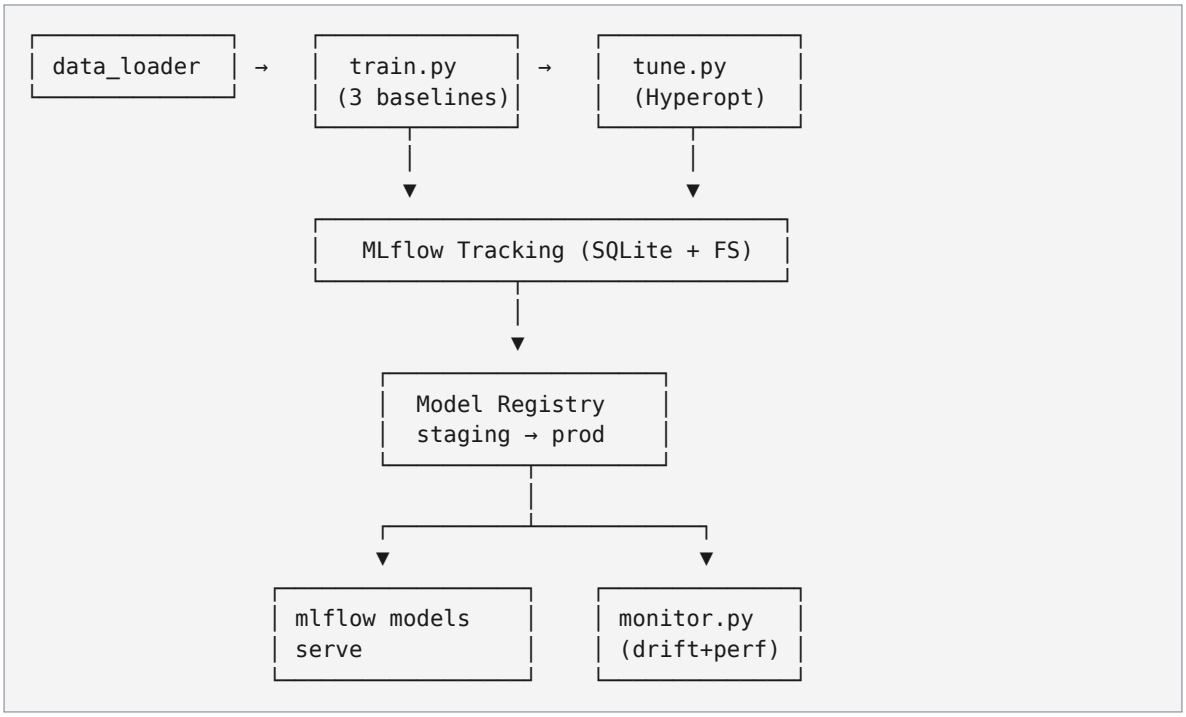
Telecommunications operators face high customer churn rates. Identifying customers likely to cancel within the next billing cycle lets the retention team intervene with targeted offers. We frame this as binary classification: predict `Churn ∈ {0, 1}` from a mix of demographic, contract, and billing features for each customer.

Dataset: IBM "Telco Customer Churn" — 7,043 customers, 20 features (15 categorical + 4 numeric + 1 customer ID), prevalence ≈ 26.5 % positive class.

Splits: stratified — 70 % train (4,929), 10 % validation (705), 20 % test (1,409).

2. System overview

The system is structured as six MLflow-managed stages:



3. Methodology and tooling

Concern	Tool
Tracking server	MLflow 3.12 with SQLite backend store, local artifact root
ML framework	scikit-learn 1.8 (Pipeline + ColumnTransformer)
Tuning	Hyperopt 0.2.7 (Tree-structured Parzen Estimator)
Model packaging	`mlflow.sklearn.log_model` with inferred signature + input example
Deployment	`mlflow models serve` loading `models:/telco-churn-classifier@production`
Drift detection	Population Stability Index + Kolmogorov–Smirnov + Evidently HTML reports

3.1 Preprocessing

A single sklearn `Pipeline` wraps preprocessing + estimator so the model artifact stored by MLflow is end-to-end:

- numeric features → median imputation + `StandardScaler`
- categorical features → most-frequent imputation + `OneHotEncoder(handle_unknown="ignore")`

The `TotalCharges` column ships with 11 blank strings (customers with `tenure=0`); these are coerced to `NaN` and filled with `0.0` in `data_loader.clean`.

4. Experiments

4.1 Baselines (Objective 1)

Three model families with default-ish parameters, each producing one MLflow run with parameters, train/val/test metrics, a confusion matrix and ROC curve as artifacts, and the fitted pipeline as a model artifact with signature.

Model	Test ROC-AUC	Test F1	Test Accuracy
LogisticRegression	**0.8426**	0.6049	0.8062
GradientBoostingClassifier	0.8390	0.5710	0.7963
RandomForestClassifier	0.8223	0.5392	0.7828

The Logistic Regression baseline is the strongest of the three out-of-the-box. This is typical for the Telco churn dataset — the predictive signal is predominantly linear (tenure, contract type, monthly charges) and the dataset is small enough that high-capacity models tend to overfit before they out-perform a regularized linear model.

4.2 Hyperparameter tuning (Objective 2)

Hyperopt TPE search over Gradient Boosting (20 trials). Each trial is a nested MLflow run under one parent `hyperopt_gradient_boosting` run, so the search history is browseable in the UI.

Search space:

Parameter	Distribution
n_estimators	quniform(50, 400, step=25)
learning_rate	loguniform(0.01, 0.3)
max_depth	quniform(2, 6)
min_samples_split	quniform(2, 20)
min_samples_leaf	quniform(1, 20)
subsample	uniform(0.6, 1.0)

Best configuration found:

```
learning_rate     = 0.0180
max_depth         = 5
min_samples_leaf  = 18
min_samples_split = 4
n_estimators      = 225
subsample         = 0.627
```

Best metrics: val ROC-AUC = 0.8672, **test ROC-AUC = 0.8457** — the tuned GB model now beats every baseline including LogReg. The dominant gain relative to the GB baseline (0.8390 → 0.8457) comes from a lower learning rate combined with stronger regularization (`min_samples_leaf=18`, `subsample=0.63`).

4.3 Model packaging and registry (Objective 5)

`src/registry.py` implements three operations:

- `register-best` — scans the `telco_churn` experiment, picks the run with the highest `test_roc_auc` (or `best_test_roc_auc` for tuning parent runs), resolves the MLflow 3 "Logged Model" UUID, and creates a new version of the registered model `telco-churn-classifier`.
- `transition <version> <stage>` — sets the modern MLflow alias (`@staging`, `@production`) **and** writes a legacy `stage` tag so the lifecycle stages the brief asks for are visible in both APIs and the UI.
- `list` — prints the current versions, their stage tags, aliases, source run IDs, and selection metrics.

After running these commands the registered model has one version:

```
{version: 2, stage_tag: 'production', aliases: 'production,staging',
 run_id: b9596f10..., score: 0.845687}
```

4.4 Deployment (Objective 3)

The production model is exposed via MLflow's built-in scoring server, as called out in Solution Guide step 5 ("MLflow's model serving capabilities"):

```
mlflow models serve \
    --model-uri "models:/telco-churn-classifier@production" \
    --port 5001 --no-conda
```

The launch script `scripts/start_mlflow_serve.sh` wraps that command and activates the project venv so the spawned gunicorn / uvicorn subprocesses inherit the right Python interpreter on PATH.

Endpoints exposed:

Endpoint	Use case
`GET /ping`	health check
`POST /invocations`	accepts `{"dataframe_split": {"columns": [...], "data": [...]}}`

Verified end-to-end: a 3-row batch from the test split returns `{"predictions": [0, 1, 0]}`. This path requires zero serving code — MLflow reconstructs the full sklearn `Pipeline` (preprocessor + estimator) from the logged artifact and routes requests through it, so the JSON record goes straight from the wire to `pipe.predict` with no manual feature engineering at inference time.

4.5 Drift monitoring (Objective 4)

`src/monitor.py` simulates three production batches against the `telco-churn-classifier@production` model. Each batch is logged as its own MLflow run in a separate `telco_churn_monitoring` experiment, so the metric time-series is browseable in the UI. The simulated batches are:

1. **clean** — random sample of test set, no perturbation
2. **feature_drift** — monthly charges shifted up by 15–35 %, contracts skewed toward Month-to-month (simulating a billing system change)
3. **concept_drift** — features unchanged but 25 % of labels flipped (the relationship between X and y has shifted, e.g. a new competitor)

Per-batch metrics logged: PSI per numeric feature, KS statistic + p-value per numeric feature, `max_psi`, a binary `drift_alert` (PSI threshold > 0.25), and full model performance (`prod_accuracy`, `prod_roc_auc`, `prod_f1`, …). An Evidently HTML report is attached as an artifact.

Results:

Scenario	max_psi	drift_alert	prod_roc_auc	prod_f1
clean	0.023	False	0.834	0.551
feature_drift	1.488	**True**	0.843	0.617
concept_drift	0.016	False	**0.654**	0.445

The third scenario is the key insight of the project: **PSI alone is blind to concept drift**. Features look identical to training, the alarm does not fire, yet model performance collapses. This is why production monitoring must combine input-distribution checks with performance metrics on labelled samples whenever ground truth is available (delayed labels, post-decision feedback, manual review

queues, etc.).

5. Discussion and insights

1. **Simpler can win on small tabular data.** The strongest out-of-the-box model was LogisticRegression; only after Hyperopt with regularization tuning did Gradient Boosting overtake it. Capacity isn't free.
2. **MLflow 3 separates "Logged Models" from runs.** The artifact path is `mlruns/<exp>/models/m-<uuid>/`, not the old `mlruns/<exp>/<run>/artifacts/model`. Registering a version using the legacy `runs:/<run>/model` URI silently succeeds but breaks on load. The registry module here resolves the `model_id` via `search_logged_models` to avoid this.
3. **Aliases > Stages.** MLflow's named-alias model (`@staging`, `@production`) makes promotion atomic and reversible. Tags are kept alongside for parity with the brief, but the serving layer reads from the alias.
4. **PSI ≠ performance.** Two monitoring signals are necessary, not one.
5. **Pipeline-as-artifact.** Logging the full sklearn `Pipeline` (preprocessor + estimator) means the scoring server does no manual preprocessing — the JSON record goes straight to `pipe.predict`, eliminating an entire class of training/serving skew bugs.

6. Reproducibility

Item	Value
Python	3.12.13
Random seeds	`RANDOM_STATE = 42` (splits + estimators), `seed=7` (monitoring batches)
Hyperopt RNG	`np.random.default_rng(42)`
Dataset hash	`data/raw/telco_churn.csv` (948 KB, 7,043 rows) downloaded from the IBM mirror on GitHub

All metrics in this report are reproduced by running the commands in `README.md` in order.

7. Limitations and future work

- Threshold tuning is not currently exposed — the scoring server uses 0.5; in production we'd pick a threshold maximizing expected retention value given the cost of a save offer.
- Class imbalance is mild (≈ 27 %) so we did not apply SMOTE / class weights; these are worth a sweep for the F1-vs-recall tradeoff.
- The drift monitoring script logs metrics but does not auto-rollback the registered model. A natural extension is to gate `@production` promotion on `prod_roc_auc` exceeding a threshold over a rolling window.

- The artifact store is local; for a multi-user setup the backend store would be Postgres and the artifact root S3 / GCS.